



CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BÀI 2: Độ quy và chia để trị

PHẦN 1



Đệ quy là gì?

Hàm đệ quy là hàm tự gọi lại chính nó trong thân hàm.

Ý tưởng:

- Chia bài toán thành các bài toán con giống hệt nhưng nhỏ hơn.
- Giải bài toán con, rồi dùng kết quả đó ghép lại để giải bài toán lớn.

Ba câu hỏi “kinh điển” khi gặp một bài toán có thể dùng đệ quy:

- Trạng thái (state) của bài toán là gì? (n, index, left/right, ...)
- Khi nào thì dừng (base case)?
- Nếu đã biết lời giải cho bài toán nhỏ hơn, dùng nó thế nào để giải bài toán.

Khung đệ quy chuẩn: Base case và Recursive case

Mọi hàm đệ quy “lành mạnh” đều có 2 phần:

- Base case:
 - Điều kiện dừng, trường hợp nhỏ nhất của bài toán.
 - Không gọi đệ quy nữa, trả về kết quả trực tiếp.
- Recursive case:
 - Gọi lại chính hàm đó với đầu vào nhỏ hơn / đơn giản hơn.
 - Dựa trên kết quả của lời gọi đệ quy để tính kết quả hiện tại.

Ví dụ 1 – Tính giai thừa:

```
def factorial(n):  
    # base case  
    if n == 0 or n == 1:  
        return 1  
  
    # recursive case  
    return n * factorial(n - 1)
```

Giải thích:

- Base case: $n = 0$ hoặc 1 thì $n! = 1$, không gọi tiếp.
- Recursive case: $n! = n \times (n-1)!$ nên ta gọi `factorial(n-1)`.

Ví dụ 2 – Tính tổng $1 + 2 + \dots + n$

Diễn giải:

- $\text{sum_to_n}(1) = 1.$
- $\text{sum_to_n}(5) = 5 + \text{sum_to_n}(4).$
- $\text{sum_to_n}(4) = 4 + \text{sum_to_n}(3).$
- $\square \dots$ cho tới base case.

```
def sum_to_n(n):  
    # base case  
    if n == 1:  
        return 1  
  
    # recursive case  
    return n + sum_to_n(n - 1)
```

Nếu quên base case thì chuyện gì xảy ra?

Nếu chọn base case sai (ví dụ $n == 0$ nhưng trong recursive case lại không bao giờ giảm về 0) thì sao?

Call stack là gì?

- Khi chương trình chạy, mỗi lần gọi hàm sẽ tạo ra một “frame” trên stack gọi là call stack.
- Hàm gọi sau nằm trên hàm gọi trước (Last In First Out).
- Với đệ quy: mỗi lần gọi chính nó sẽ “chồng thêm” một frame mới.
- Khi hàm trả về, frame trên cùng được pop ra, trả kết quả về cho frame phía dưới.
- Đệ quy không phải “phép màu”, nó chỉ là cơ chế gọi hàm chồng lên nhau bằng call stack mà thôi.

Minh họa call stack với factorial(3)

Diễn biến:

- Gọi factorial(3):
 - Chưa biết kết quả, cần factorial(2).
- Gọi factorial(2):
 - Chưa biết kết quả, cần factorial(1).
- Gọi factorial(1):
 - Base case → trả về 1.

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(3))
```


Quá trình trả về:

- factorial(1) trả về 1 cho factorial(2).
- factorial(2) tính $2 * 1 = 2$, trả về 2 cho factorial(3).
- factorial(3) tính $3 * 2 = 6$, trả về 6 cho hàm main.

Chúng ta sẽ có 2 chiều:

- Pha “đi xuống”: gọi thêm, stack dày lên.
- Pha “đi lên”: return ngược, stack mỏng dần.

Stack overflow trong đệ quy

```
def infinite_recursion():  
    return infinite_recursion()  
  
infinite_recursion()
```

- Không có base case → gọi mãi mãi.
- Call stack ngày càng dày, tới giới hạn sẽ bị lỗi RecursionError (Python).
- Vì vậy: luôn phải có base case đảm bảo chắc chắn sẽ được đạt tới sau hữu hạn bước.

Binary search – Review ý tưởng chia để trị

Ý tưởng chia để trị:

- So sánh target với phần tử ở giữa.
- Nếu $\text{target} < \text{mid} \rightarrow$ chỉ còn khả năng nằm bên trái.
- Nếu $\text{target} > \text{mid} \rightarrow$ chỉ còn khả năng nằm bên phải.
- Nếu bằng $\rightarrow$ tìm được.

Độ phức tạp:

- Mỗi bước “bỏ đi” một nửa mảng.
- Số bước $\approx \log_2(n) \rightarrow O(\log n)$.

Binary search – phiên bản lặp (iterative)

- left và right là biên đang xét.
- Mỗi vòng lặp, khoảng tìm kiếm giảm một nửa.

```
def binary_search_iterative(arr, target):  
    left = 0  
    right = len(arr) - 1  
  
    while left <= right:  
        mid = (left + right) // 2  
  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            left = mid + 1 # bỏ nửa trái  
        else:  
            right = mid - 1 # bỏ nửa phải  
  
    return -1 # không tìm thấy
```

Binary search – phiên bản đệ quy

```
def binary_search_recursive(arr, target, left, right):  
    # base case: khoảng tìm kiếm rỗng  
    if left > right:  
        return -1  
  
    mid = (left + right) // 2  
  
    if arr[mid] == target:  
        return mid  
    elif arr[mid] < target:  
        # recursive case 1: tìm bên phải  
        return binary_search_recursive(arr, target, mid + 1, right)  
    else:  
        # recursive case 2: tìm bên trái  
        return binary_search_recursive(arr, target, left, mid - 1)
```

Phân tích đệ quy binary search

- Base case: $\text{left} > \text{right} \rightarrow$ không còn phần tử nào để tìm, trả về -1.
- Recursive case:
 - Nếu $\text{arr}[\text{mid}] < \text{target} \rightarrow$ gọi lại ở nửa phải.
 - Nếu $\text{arr}[\text{mid}] > \text{target} \rightarrow$ gọi lại ở nửa trái.
- Mỗi lần gọi chỉ giữ lại một nửa mảng $\rightarrow$ sau mỗi bước, kích thước bài toán giảm từ n xuống $n/2, n/4, \dots$
- Số lần gọi đệ quy: $\approx \log_2(n)$.
- Độ phức tạp thời gian: $O(\log n)$.

So sánh iterative vs recursive – binary search

Ưu điểm đệ quy:

- Mã ngắn gọn, bám sát ý tưởng chia để trị.
- Dễ chứng minh bằng quy nạp.

Nhược điểm:

- Tốn thêm stack frame $\rightarrow O(\log n)$ không gian.
- Nếu code không cẩn thận có thể gây stack overflow với ngôn ngữ có giới hạn call stack nhỏ.

“Đệ quy đúng nhưng chậm” – Ví dụ Fibonacci

Định nghĩa dãy Fibonacci:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ với $n \geq 2$

Đệ quy “ngây thơ” như code bên cạnh

Đặc điểm:

- Base case: $n = 0$ hoặc 1 .
- Recursive case: 2 lời gọi đệ quy cho mỗi lần.

```
def fib_slow(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
  
    return fib_slow(n - 1) + fib_slow(n - 2)
```


Cây đệ quy của Fibonacci

Ví dụ $n = 5$:

- $\text{fib}(5) = \text{fib}(4) + \text{fib}(3)$
- $\text{fib}(4) = \text{fib}(3) + \text{fib}(2)$
- $\text{fib}(3) = \text{fib}(2) + \text{fib}(1)$
- $\square \dots$

Nếu vẽ cây lời gọi:

- $\text{fib}(3)$ xuất hiện nhiều lần.
- $\text{fib}(2)$ xuất hiện nhiều lần.

Ý nghĩa:

- Cùng một giá trị n nhưng tính lại nhiều lần.
- Độ phức tạp thời gian: xấp xỉ $O(2^n)$ – tăng rất nhanh, $n \approx 40-45$ là bắt đầu lag.

Ý tưởng “đệ quy có nhớ” (memoization)

Mục tiêu:

- Vẫn dùng đệ quy (code dễ đọc).
- Tránh tính lại những giá trị đã tính rồi.

Khái niệm:

- Memoization = “Nhớ” lại kết quả của các lời gọi trước trong một bảng (thường là dict hoặc list).
- Lần sau gặp lại cùng tham số, chỉ cần tra bảng, không cần tính lại.

Fibonacci với memoization

- memo là dict: key = n, value = F(n).
 - Mỗi n chỉ được tính tối đa 1 lần.
 - Số trạng thái cần tính là $n+1$ ($0 \rightarrow n$).
 - Mỗi trạng thái xử lý $O(1)$
- (cộng 2 số) $\rightarrow$ tổng thời gian $O(n)$.

```
def fib_memo(n, memo=None):  
    if memo is None:  
        memo = {}  
  
    # nếu đã tính rồi thì trả về luôn  
    if n in memo:  
        return memo[n]  
  
    # base cases  
    if n == 0:  
        memo[0] = 0  
    elif n == 1:  
        memo[1] = 1  
    else:  
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)  
  
    return memo[n]
```

So sánh “đệ quy chậm” vs “đệ quy có nhớ”

fib_slow(n):

- Độ phức tạp thời gian: $O(2^n)$.
- Số lời gọi đệ quy cực nhiều, trùng lặp.

fib_memo(n):

- Độ phức tạp thời gian: $O(n)$.
- Vẫn là đệ quy, nhưng không lãng phí.

```
import time

start = time.time()
a = fib_slow(35)
end = time.time()
print(f"fib_slow: {end - start:.4f} giây")

start = time.time()
b = fib_memo(35)
end = time.time()
print(f"fib_memo: {end - start:.4f} giây")

fib_slow: 2.5081 giây
fib_memo: 0.0001 giây
```

Quy tắc nhận diện “đệ quy chậm”

Một số dấu hiệu:

- Mỗi lời gọi đệ quy lại gọi hai (hoặc nhiều) lời gọi khác với tham số “gần giống nhau” (ví dụ $n-1$ và $n-2$).
- Cùng một trạng thái có thể được tính lại nhiều lần.
- Dễ vẽ thành cây đệ quy có rất nhiều node trùng lặp.

Khi thấy các dấu hiệu trên, hãy nghĩ tới:

- Memoization (top-down).
- Hoặc chuyển sang bottom-up (dùng vòng lặp, bảng).

PHẦN 2



Chia để trị

Là một chiến lược dùng đệ quy để giải bài toán

Ba bước cốt lõi:

- Divide (chia): Chia bài toán lớn thành nhiều bài toán con nhỏ hơn
- Conquer (trị): Giải đệ quy các bài toán con (hoặc giải trực tiếp nếu đủ nhỏ)
- Combine (kết hợp): Ghép kết quả các bài toán con thành lời giải bài toán lớn

Ví dụ kinh điển:

- Merge Sort
- Quick Sort

Merge Sort

Ý tưởng Merge Sort:

- Divide: Chia mảng thành 2 nửa (trái và phải)
- Conquer: Sắp xếp đệ quy 2 nửa đó
- Combine: Trộn (merge) 2 nửa đã sắp xếp thành mảng hoàn chỉnh

Đặc điểm:

- Thuật toán ổn định (stable sort)
- Độ phức tạp: $O(n \log n)$ trong mọi trường hợp
- Tốn thêm bộ nhớ $O(n)$ để lưu mảng tạm

Ví dụ sắp xếp: [38, 27, 43, 3, 9, 82, 10]

Pha Divide (chia đôi liên tục): Chia đến khi mỗi mảng chỉ còn 1 phần tử (đã sắp xếp tự nhiên).

```
[38, 27, 43, 3, 9, 82, 10]
      ↓ chia
[38, 27, 43, 3]      [9, 82, 10]
      ↓              ↓
[38, 27] [43, 3]    [9, 82] [10]
      ↓       ↓       ↓       ↓
[38] [27] [43] [3]  [9] [82] [10]
```

Pha Conquer & Combine (trộn từng cặp):

$[38] \ [27] \rightarrow [27, 38]$

$[43] \ [3] \rightarrow [3, 43]$

$[9] \ [82] \rightarrow [9, 82]$

$[10] \rightarrow [10]$

$[27, 38] + [3, 43] \rightarrow [3, 27, 38, 43]$

$[9, 82] + [10] \rightarrow [9, 10, 82]$

$[3, 27, 38, 43] + [9, 10, 82] \rightarrow [3, 9, 10, 27, 38, 43, 82]$

Merge Sort – Code Python

```
def merge_sort(arr):
    # Base case: mảng 0 hoặc 1 phần tử đã sắp xếp
    if len(arr) <= 1:
        return arr

    # Divide: chia đôi mảng
    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    # Conquer: đệ quy sắp xếp 2 nửa
    left_sorted = merge_sort(left_half)
    right_sorted = merge_sort(right_half)

    # Combine: trộn 2 nửa đã sắp xếp
    return merge(left_sorted, right_sorted)
```

```
def merge(left, right):
    """Trộn 2 mảng đã sắp xếp thành 1 mảng sắp xếp"""
    result = []
    i = j = 0

    # So sánh và chọn phần tử nhỏ hơn
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Thêm các phần tử còn lại (nếu có)
    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

Phân tích Merge Sort

Độ phức tạp thời gian:

- Divide: Chia đôi mảng $\rightarrow O(1)$ (chỉ tính chỉ số giữa)
- Conquer: Gọi đệ quy 2 nửa $\rightarrow 2 \times T(n/2)$
- Combine: Trộn 2 mảng n phần tử $\rightarrow O(n)$

Công thức đệ quy:

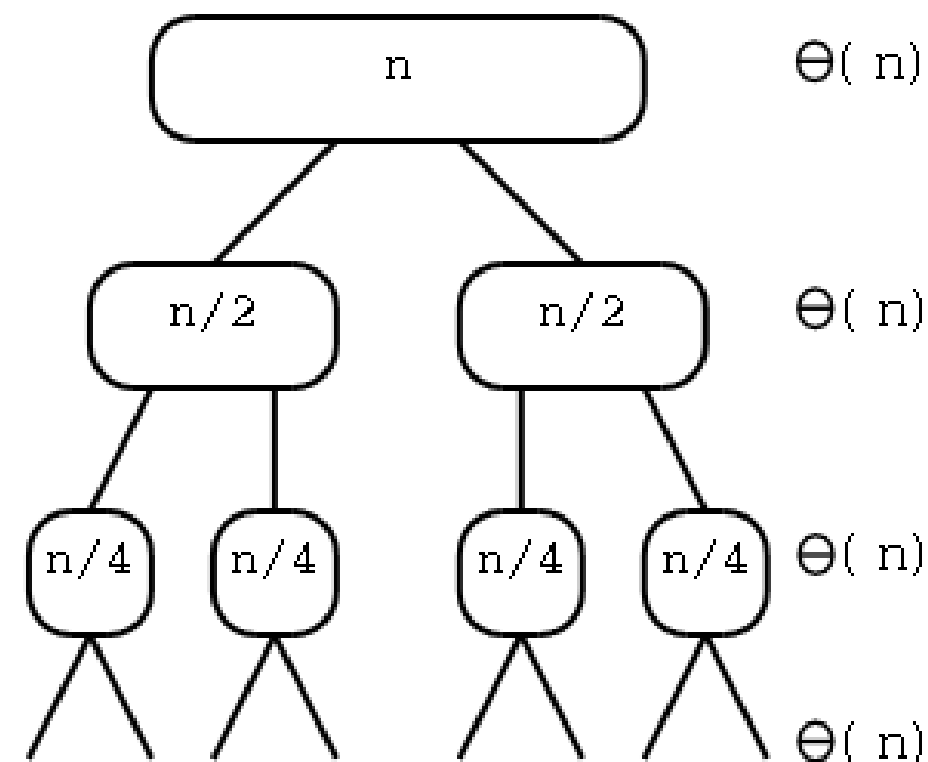
$$T(n) = 2 \times T(n/2) + O(n)$$

Giải bằng Master Theorem hoặc vẽ cây đệ quy:

- Chiều cao cây: $\log_2(n)$
- Mỗi tầng tốn $O(n)$ để merge
- Tổng: $O(n) \times \log(n) = O(n \log n)$

Độ phức tạp không gian: $O(n)$ do cần mảng tạm khi merge

Best/Worst/Average case: Đều $O(n \log n)$



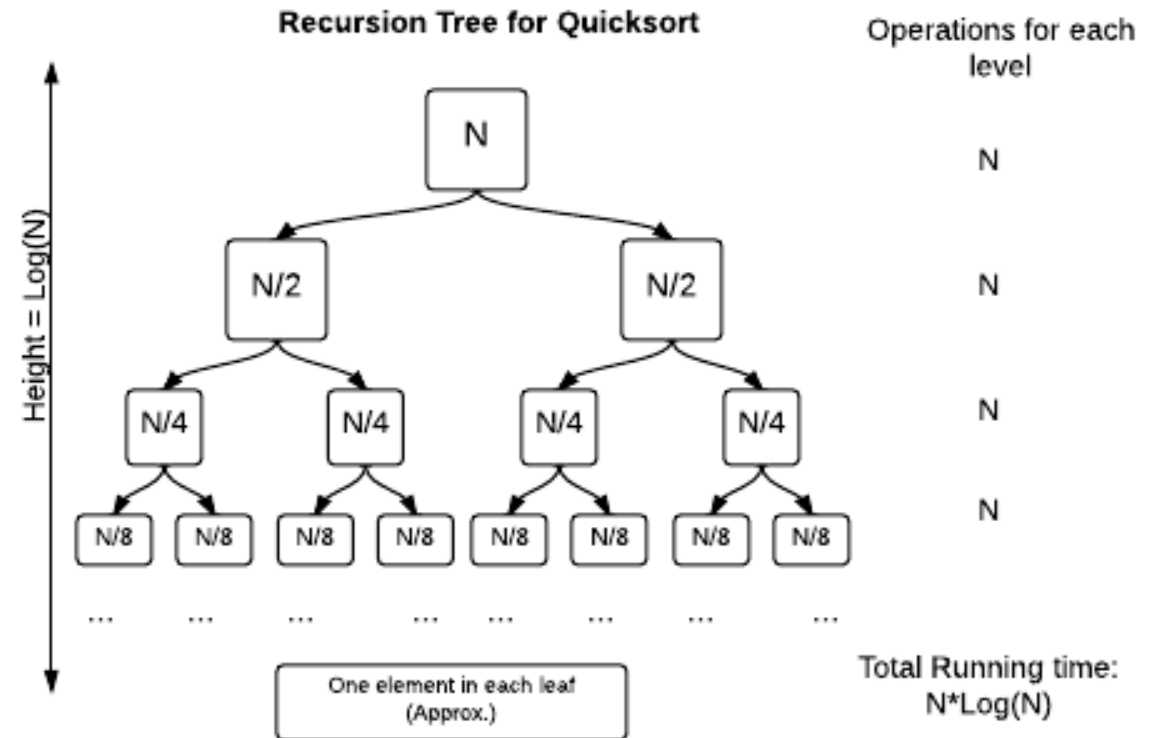
Quick Sort – Ý tưởng

Chiến lược:

- Chọn pivot: Chọn 1 phần tử làm "chốt" (thường chọn phần tử cuối)
- Partition: Sắp xếp lại sao cho:
 - Các phần tử $<$ pivot nằm bên trái
 - Các phần tử $\geq$ pivot nằm bên phải
- Pivot nằm đúng vị trí cuối cùng của nó
- Đệ quy: Sắp xếp đệ quy 2 phần trái và phải của pivot

Đặc điểm:

- Không ổn định (unstable)
- Best/Average case: $O(n \log n)$
- Worst case: $O(n^2)$ nếu pivot chọn tệ (mảng đã sắp xếp, pivot là min/max)
- Không tốn bộ nhớ phụ (sort in-place) $\rightarrow O(1)$ không gian (không tính stack)



Quick Sort – Minh họa Partition

Ví dụ: [7, 3, 9, 8, 5, 1], chọn pivot = 1 (phần tử cuối)

Bước partition:

```
Ban đầu: [7, 3, 9, 8, 5, 1]  pivot = 1
           ^               ^
           left            right
```

So sánh left với pivot:

- 7 > 1 → giữ nguyên
- Tìm right: 5 > 1, 8 > 1, 9 > 1, 3 > 1, 7 > 1
- Không swap được → pivot đã ở đầu

Kết quả: [1, 3, 9, 8, 5, 7]

```
           ^
           pivot đúng vị trí (index 0)
```


Thực tế với pivot tốt hơn, ví dụ pivot = 5:

```
[7, 3, 9, 8, 5, 1]  pivot = 5
  ^               ^
left             right

Partition → [3, 1, 5, 8, 9, 7]
              ^
              pivot tại index 2

Đệ quy trái: [3, 1]
Đệ quy phải: [8, 9, 7]
```

Quick Sort – Code Python

```
def quick_sort_inplace(arr, low, high):  
    if low < high:  
        # Tìm vị trí pivot sau partition  
        pi = partition(arr, low, high)  
  
        # Đệ quy 2 phần  
        quick_sort_inplace(arr, low, pi - 1)  
        quick_sort_inplace(arr, pi + 1, high)
```

```
def partition(arr, low, high):  
    pivot = arr[high] # Chọn phần tử cuối làm pivot  
    i = low - 1 # Chỉ số phần tử nhỏ hơn pivot  
  
    for j in range(low, high):  
        if arr[j] < pivot:  
            i += 1  
            arr[i], arr[j] = arr[j], arr[i] # Swap  
  
    # Đặt pivot vào đúng vị trí  
    arr[i + 1], arr[high] = arr[high], arr[i + 1]  
    return i + 1
```

Phân tích Quick Sort

Best case: Pivot luôn chia đôi mảng

- $T(n) = 2 \times T(n/2) + O(n)$
- Kết quả: $O(n \log n)$

Average case: Pivot "tương đối tốt"

- Kết quả: $O(n \log n)$

Worst case: Pivot luôn là min/max (mảng đã sắp xếp)

- $T(n) = T(n-1) + O(n)$
- Kết quả: $O(n^2)$

Cách tránh worst case:

- Chọn pivot ngẫu nhiên (randomized quicksort)
- Chọn pivot là median của 3 phần tử (first, mid, last)

Độ phức tạp không gian:

- Phiên bản in-place: $O(1)$ (không tính stack)
- Call stack: $O(\log n)$ trung bình, $O(n)$ worst case

So sánh Merge Sort vs Quick Sort

Tiêu chí	Merge Sort	Quick Sort
Best case	$O(n \log n)$	$O(n \log n)$
Average case	$O(n \log n)$	$O(n \log n)$
Worst case	$O(n \log n)$	$O(n^2)$
Không gian	$O(n)$	$O(1)$ (in-place)
Ổn định	Có	Không
Ứng dụng	Dữ liệu lớn, cần stable	Dữ liệu vừa, cần nhanh

Định lý (Master theorem)

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Trong đó:

- a: số lời gọi đệ quy
- b: tỉ lệ chia nhỏ bài toán
- f(n): chi phí chia và kết hợp

Ba trường hợp chính:

- Nếu $f(n) = O(n^c)$ với $c < \log_b a$ thì $T(n) = \theta(n^{\log_b a})$
- Nếu $f(n) = \theta(n^{\log_b a})$ thì $T(n) = \theta(n^{\log_b a} \log n)$
- Nếu $f(n) = \Omega(n^c)$ với $c > \log_b a$ thì $T(n) = \theta(f(n))$

Áp dụng Master Theorem

Ví dụ 1: Merge Sort

$$T(n) = 2 \times T(n/2) + O(n)$$

$$a = 2, b = 2, f(n) = n$$

$$\log_b a = \log_2 2 = 1$$

$$f(n) = n^1 \rightarrow c = 1 = \log_b a$$

Trường hợp 2: $T(n) = \theta(n \log n)$

Ví dụ 2: Binary Search

$$T(n) = 1 \times T(n/2) + O(1)$$

$$a = 1, b = 2, f(n) = 1$$

$$\log_b a = \log_2 1 = 0$$

$$f(n) = n^0 = 1 \rightarrow c = 0 = \log_b a$$

Trường hợp 2: $T(n) = \theta(\log n)$



Kết thúc